



Automated Remediation in CI/CD: Design and Control of an AI-based Code Repair Agent

Authors:

Ahmad Darwich
Mouaz Naji

ahda23@student.bth.se
moap23@student.bth.se

Supervisor:

To be determined

Date: 2026-02-05

Background

Modern software engineering relies heavily on Continuous Integration and Continuous Deployment (CI/CD) pipelines to ensure rapid and reliable software delivery. However, pipelines frequently fail due to issues such as failing tests, build errors, or small code regressions. Today, developers often need to manually inspect logs, identify the root cause, apply a fix, rerun tests, and then submit the changes again. This manual process creates a time-consuming bottleneck in software delivery and diverts engineering resources from feature development. To address these inefficiencies, the integration of AI-based agents for automated remediation is becoming a viable solution. By leveraging Generative AI, these agents can potentially detect failures, interpret logs, and propose code fixes autonomously.

Focus area

Our scope will be aimed at investigating how an AI-based "code repair agent" can be designed and controlled to support the CI/CD process effectively. The study will provide a holistic view of the architectural requirements for such an agent, moving beyond simple code generation to reliable system integration. Through an analysis of necessary control mechanisms such as quality gates, approval steps, and audit trails we can provide a thorough evaluation of how to maintain code quality and safety while automating repairs.

Motive and Value

Time-to-recovery is a critical metric for DevOps performance. For software organizations, the ability to automatically resolve routine pipeline failures translates to significantly reduced downtime and faster feedback loops. The value of this thesis lies in establishing a trusted framework for autonomous agents in CI/CD, thus leading to increased developer productivity and a reduction in manual, repetitive maintenance tasks.

References (Preliminary)

1. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
2. Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). Wiley.
3. Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

Empirical Implementation (High-Level Description):

The empirical implementation will focus on the following:

- **Architecture Design:** We will develop a detailed architecture blueprint for the agent, including practical control mechanisms such as quality gates (linting, security scanning) and retry limits.
- **Data Collection (Interviews):** We will conduct 1–3 short interviews (30–45 minutes) with industry engineers or tech leads to understand real-world failure handling and the requirements for trusting an autonomous agent.
- **Validation:** Feedback on the proposed architecture will be gathered from industry experts to ensure it addresses real risks and operational constraints.
- **Note:** We do not need access to proprietary source code. The collaboration with the industry partner will focus on expert input regarding failure patterns, risks, and process requirements.

Preliminary Goals:

- Identify the critical risks and necessary "quality gates" for deploying autonomous repair agents in a production pipeline.
- Demonstrate how an AI agent can interpret build logs and test results to generate targeted, safe code fixes.
- Develop a structured blueprint for auditing and controlling AI agents to prevent infinite loops or unsafe changes.

Preliminary Research Questions:

1. How should an AI code repair agent be designed and controlled in a CI/CD pipeline to accurately detect and fix build failures based on logs?
2. What control mechanisms (e.g., human-in-the-loop, security scanning, audit trails) are required to ensure the agent does not introduce unsafe changes or technical debt?
3. What are the primary barriers to trust for autonomous remediation agents among software engineers, and how can architectural design mitigate these concerns?